

Лабораторна робота №3.

Проектування функції винагороди та політ до заданої точки

25 серпня 2026 р.

Мета

Навчитися будувати складену функцію винагороди для квадрокоптера й вимірювати внесок кожного доданка абляцією. Навчити політику польоту до заданої точки та розпізнавати погану винагороду за числами в логах.

Склад бригади і розподіл ролей

Робота виконується бригадою з трьох осіб. Кожен учасник відповідає за свою підсистему й захищає її окремо.

- **Учасник А — середовище і винагорода.** Реалізує середовище польоту до точки й складену винагороду, яка повертає і суму, і кожен доданок окремо; забезпечує вимикання будь-якого доданка нульовою вагою.
- **Учасник В — абляція.** Проводить шість абляційних експериментів, два з них із повним навчанням політики, і фіксує, що саме ламається в поведінці при вимиканні кожного доданка.
- **Учасник С — зіпсована винагорода.** Досліджує вагу штрафу за різкість керування й навмисно псує винагороду двома способами, показуючи, як політика експлуатує формулювання.

Інтеграційний крок виконують разом: усі запуски зводять в одну таблицю й формулюють, без яких доданків задача не розв'язується, а які змінюють лише манеру польоту.

Теоретичні відомості

Алгоритм максимізує суму винагород за епізод, і тільки її. Чого немає у винагороді, того для політики не існує, а записане з неправильною вагою виконується рівно настільки, наскільки вигідно. Винагорода і є постановкою задачі, тому помилку в ній не виправить довше навчання.

Стандартний набір доданків для польоту до точки має вигляд

$$r_t = -w_d \|p^* - p_t\| - w_\omega \|\omega_t\| - w_\theta (1 - R_{33}) - w_a \|a_t - a_{t-1}\| - w_c \mathbb{K}_{\text{crash}} + w_l \quad (1)$$

де p^* , p_t — цільова й поточна позиція, м; ω_t — кутова швидкість, рад/с; R_{33} — елемент матриці повороту (при рівному польоті $R_{33} = 1$, тому $(1 - R_{33})$ — відхилення від горизонту); a_t — дія: 4 числа СТБР (тяга і три кутові швидкості); $\mathbb{K}_{\text{crash}}$ — аварія: удар об підлогу чи стелю або нахил понад 1,2 рад. Базові ваги: $w_d = 1,00$, $w_\omega = 0,05$, $w_\theta = 0,50$, $w_a = 0,02$, $w_c = 10,0$, $w_l = 0,10$.

Головний внесок дає відстань до цілі, близько 74 % модуля суми. Решта доданків задає лише *манеру* польоту. Це формувальні доданки (**reward shaping**): у формі $F = \gamma\Phi(s_{t+1}) - \Phi(s_t)$ вони не змінюють оптимуму. Штраф за аварію коштує близько 100 кроків польоту, тобто двох секунд. Без бонусу за життя всі доданки від’ємні, і найвигідніше впасти одразу.

Погану винагороду видають три ознаки:

- **локальний оптимум**: плато далеко від очікуваного, при $w_c = 100$ апарат висить під стелею;
- **уникання епізоду**: епізод коротшає, бо бонус за життя замалий;
- **експлуатація щілини (reward hacking)**: сума росте, а задача не виконується. При $w_l = 1,0$ вигідніше простояти 400 кроків біля старту (+400), ніж летіти з 2 м (−150).

За самою сумою жодної з них не видно. Потрібен розклад на доданки.

Очікуваний ефект вимикання доданків

Вимкнено	Що станеться з політикою	Як це видно в числах
наближення, w_d	задачі немає	відстань не спадає між епізодами
кутова швидкість, w_ω	крутиться заради маневру	середня $\ \omega\ $ понад 3 рад/с
нахил, w_θ	різкі нахили	аварій більше, епізод коротший
різкість дій, w_a	«смикає» мотори	ривок дії більший у 3–5 разів
аварія, w_c	летить напролом	аварій 3–5 із 5, епізод < 120 кроків
бонус за життя, w_l	обірвати епізод вигідно	сума росте, епізод коротшає

Політика отримує 18 чисел про власний стан. Цей самий склад успадковує робота №4, де до нього додається блок сусідів, тому зафіксуйте його точно.

Блок	Компоненти	Розмір	Одиниці
Вектор на ціль	$p_{\text{goal}} - p$	3	м
Лінійна швидкість	v_x, v_y, v_z	3	м/с
Орієнтація	матриця повороту R	9	—
Кутова швидкість	p, q, r	3	рад/с
Разом		18	

Орієнтацію подають саме матрицею повороту, а не кутами Ейлера: кути мають розриви на $\pm 180^\circ$, і мережа на них зривається.

Корисні посилання для ознайомлення:

- [Batra та ін. Decentralized Control of Quadrotor Swarms](#) — звідки взято склад винагороди;
- [Ng, Harada, Russell. Policy Invariance Under Reward Transformations](#) — коли shaping не змінює оптимуму;
- [Amodei та ін. Concrete Problems in AI Safety](#) — розділ про reward hacking;
- [Kaufmann та ін. Champion-level drone racing using deep RL](#) — винагорода в реальних перегонах.

Порядок виконання

1. **[разом]** Підготувати середовище: Python 3.10–3.12, `numpy` ≥ 1.24 , `matplotlib` ≥ 3.7 , `gym-pybullet-drones` 2.0 з `pybullet` $\geq 3.2.5$, `stable-baselines3` 2.2–2.3.
2. **[А]** Реалізувати власне середовище польоту до точки й складену винагороду за формулою (1) з шести доданків. Функція має повертати і суму, і кожен доданок окремо, накопичений за епізод — без цього розкладу решту пунктів не виконати. Передбачити вимикання кожного доданка нульовою вагою. Як опорну політику написати простий ПІД-регулятор положення.
3. **[А]** Виконати опорний прогін на 5 епізодів із ціллю $(0,0; 0,0; 1,0)$ м і базовими вагами. Очікується: доданок наближення $-120 \dots -130$ за епізод, бонус за життя рівно $+40,0$, сума близько -85 , епізод 400 кроків із 400, аварій нуль, відстань $\leq 0,10$ м. Зберегти ці числа як опорні.
4. **[В]** Провести шість абляційних експериментів: у кожному вимкнути рівно один доданок, решту лишити базовими. Бонус за життя й штраф за різкість дослідити з повним навчанням політики (бюджет 300 тис. кроків), решту чотири прогнати з опорною ПІД-політикою. Зафіксувати кінцеву відстань, довжину епізоду, аварії, ривок дії, розклад винагороду й одним реченням записати, що саме зламалося в поведінці.
5. **[С]** Прогнати три значення $w_a = 0; 0,02; 0,20$ — змінюючи лише цю вагу. Побудувати три траєкторії $z(t)$ на одному графіку, окремо три графіки тяги $u(t)$ і таблицю: w_a , середній ривок, кінцева відстань, час виходу в зону 0,10 м, внесок доданка різкості. Очікується: при $w_a = 0$ ривок 0,05–0,08; при $w_a = 0,02$ ривок падає вдвічі, а час польоту зростає на 5–10 %; при $w_a = 0,20$ політ помітно повільніший.
6. **[С]** Зіпсувати винагороду щонайменше двома способами з трьох: завеликий бонус за життя $w_l = 1,0$; прибралий штраф за відстань, замінений власним бонусом за наближення, який політика вміє «накручувати»; занадто дорога аварія $w_c = 100$. Для кожного випадку навести розклад винагороду за доданками, пояснити, *як саме* політика експлуатує формулювання, і вказати, яку вагу і в який бік треба змінити.
7. **[разом]** Звести всі 11–12 запусків в одну таблицю: номер, зміна відносно опорного, сума за епізод, кінцева відстань, аварій із 5, ривок дії. За нею сформулювати висновок: без яких доданків задача не розв’язується, які змінюють лише манеру польоту, а які можна прибрати без наслідків. Висновки — 3–5 тверджень із числами.
8. **[разом]** **Оформити ноутбук.** Зібрати роботу в один Jupyter-ноутбук: код кожного учасника у своїх комірках із короткими коментарями там, де це доречно. Обов’язково всередині: опорний прогін із розкладом винагороду за доданками, шість рядків абляції з описом «що зламалося», графіки $z(t)$ і $u(t)$ для трьох значень w_a , розклад двох зіпсованих винагород і зведену таблицю всіх запусків, а також висновки — три-п’ять перевірюваних тверджень із числами. Ноутбук має виконуватися від початку до кінця без ручних правок; у першій комірці вкажіть, хто з учасників за які комірки відповідав.

Контрольні запитання

1. Чому винагорода i є постановкою задачі?
2. Що станеться з політикою, якщо всі доданки від’ємні?
3. Чому відхилення від горизонту подають через R_{zz} , а не через кут?

4. Скільком крокам польоту відповідає штраф за аварію в 10 одиниць?
5. Що таке reward shaping і коли він не змінює оптимуму?
6. Чим небезпечне надто складне формулювання винагороди?
7. Винагорода росте, але апарат не летить до цілі: що перевірите?
8. Чому при зростанні w_a з 0,02 до 0,20 страждає час, а не точність?
9. Навіщо абляція, якщо якість вимірюється сумою винагороди?
10. Політика висить на місці. Яку вагу зміните першою і в який бік?